Rapport de gestion de projet

Projet étudiant TP_Agile — Méthode Scrumban

Application web multi-agents IA pour la planification agile

Cours	TP Méthode Agile
Équipe	5 membres — initiales PP, F, AR, NS, YL (hypothèse de correspondance avec les comptes Git)
Méthode	Scrumban (Scrum + Kanban)
Outils	GitHub, Trello, Vite/React (frontend)
Période couverte	Octobre 2026 (d'après l'historique Git)

Sommaire

1. Introduction

2. Présentation du projet et objectifs

3. Méthode Scrumban : choix, organisation et règles

4. Organisation de l'équipe et répartition des rôles

5. Analyse de l'état d'avancement

6. Gestion du backlog et priorisation

7. Suivi via Git/GitHub

8. Rituels et pratiques d'équipe

9. Risques et difficultés identifiés

10. Points forts, axes d'amélioration et recommandations

11. Conclusion

1. Introduction

Ce rapport présente la gestion du projet étudiant TP_Agile, un travail pratique de méthode agile. Le livrable principal est une application web interactive capable de transformer une idée métier en User Stories, en backlog priorisé et en propositions de sprints, grâce à une architecture multi-agents IA. La méthode retenue combine Scrum et Kanban, d'où l'appellation Scrumban.

Le rapport suit un plan en onze sections : présentation du projet, méthode Scrumban, organisation de l'équipe, état d'avancement, backlog, suivi Git, rituels, risques, recommandations, puis conclusion.

2. Présentation du projet et objectifs

2.1 Contexte

Le projet TP_Agile s'inscrit dans un cours de méthodes agiles. L'équipe doit concevoir une application qui assiste un porteur de projet dans la phase initiale de cadrage : transformer un brief en langage naturel en un plan exécutable (User Stories, backlog, sprint).

2.2 Objectifs fonctionnels (d'après le README)

Selon la section 2 du README, l'application doit permettre à un utilisateur de :

- décrire un projet en langage naturel,
- dialoguer avec un agent Product Owner IA pour clarifier le besoin,
- obtenir une synthèse structurée des besoins,
- générer automatiquement des User Stories,
- construire un backlog priorisé,
- identifier les dépendances entre les tâches,
- estimer leur charge et les prioriser,
- recevoir des propositions de sprint.

2.3 Architecture multi-agents

Le README décrit cinq composants :

- Agent 1 — Product Owner IA : comprend le besoin métier, dialogue avec l'utilisateur, identifie les informations manquantes (fonctionnalités, utilisateurs cibles, contraintes techniques, nombre de développeurs, budget, date limite) et clarifie les ambiguïtés.
- Agent 2 — Générateur de User Stories : produit les stories à partir des informations validées, vérifie les doublons.
- Agent 3 — Générateur de backlog : décompose les User Stories en tâches pertinentes.
- Agent 4 — Agent Scrum : analyse la valeur métier, évalue l'importance fonctionnelle, identifie les dépendances, estime la complexité et la charge, ordonne les priorités, justifie ses décisions.
- Orchestrateur central : pilote l'enchaînement des agents, vérifie la validité des sorties, gère les erreurs et les nouvelles tentatives.

2.4 Interface web

L'application prévoit trois vues : backlog (ordre de priorité, dépendances, estimations), Kanban (À faire / En cours / Terminé) et sprints (charge, capacité, blocages).

3. Méthode Scrumban : choix, organisation et règles

3.1 Pourquoi Scrumban

Scrumban combine deux approches complémentaires :

- Scrum apporte le cadre formel : rôles (Product Owner, Scrum Master, équipe de développement), rituels (Daily, Sprint Planning, Review, Retrospective) et un engagement de livraison par incréments.
- Kanban apporte la fluidité visuelle, la limitation du WIP (Work In Progress) et le flux tiré, qui évitent la sur-charge et rendent l'avancement transparent.

Dans un contexte de TP court avec une architecture complexe, Scrumban évite le rythme trop rigide de Scrum pur (peu adapté à des incréments incertains) tout en conservant la discipline nécessaire pour tenir les délais.

3.2 Organisation du board

Il y a cinq colonnes et des étiquettes de couleur pour la priorité. Le tableau suivant récapitule la structure :

Colonne	Sens / rôle Scrumban
Backlog	Idées, idées à qualifier, demandes non encore planifiées. Pas de WIP strict.
À faire (sprint)	Tirées dans le sprint en cours, prêtes à être démarrées. Flux tiré : on ne tire que quand une carte « En cours » se libère.
En cours	WIP limité - un développeur ne devrait pas avoir plus d'une carte à la fois.
À review	Code terminé, en attente de revue / intégration. C'est ici que se forme le goulot.
Terminé	Carte validée, intégrée, prête à livrer.

3.3 Limites WIP suggérées

Pour une équipe de cinq personnes, voici des limites WIP indicatives qui amélioreraient le flux :

Colonne	WIP suggéré	Justification
Backlog	Pas de WIP	Le backlog est un réservoir, pas un flux.
À faire (sprint)	3 à 5 cartes	Un réservoir court prêt à être tiré, sans saturer.
En cours	5 (1 par personne)	Une seule carte active par développeur.
À review	2 à 3	Pour éviter un empilement.
Terminé	Pas de WIP	File d'archive.

3.4 Étiquettes de priorité

Les étiquettes observées sur la capture Trello utilisent un code couleur intuitif :

- Rouge - priorité haute : bloquant, doit être traité immédiatement.
- Orange - priorité moyenne : important mais pas critique, planifié dans le sprint.
- Vert - priorité basse : amélioration, nice-to-have, traité quand la capacité le permet.

4. Organisation de l'équipe et répartition des rôles

L'équipe compte cinq membres dont un product owner et un scrum master (Pasqualini Pierre) et 4 developpeurs (Lodi Ylan, Morpelli Fabien, Sauli Nicolas, Renucci Alexandre)

5. Gestion du backlog et priorisation

5.1 Critères de priorisation

L'agent Scrum applique : valeur métier, importance fonctionnelle, dépendances et date limite. Ces critères doivent être utilisés manuellement par l'équipe pour les cartes backlog, en attendant que l'agent Scrum les applique automatiquement.

- Valeur métier : impact utilisateur ou valeur pédagogique du livrable.
- Importance fonctionnelle : dépendance avec d'autres fonctionnalités (ex. connexion avant tests).
- Risques techniques : tokens, 429.

5. Rituels et pratiques d'équipe

Les rituels Scrumban combinent des éléments Scrum et Kanban. Les pratiques suivantes sont recommandées et, pour certaines, déjà observables :

5.2 Sprint Planning

L'équipe sélectionne dans le backlog un ensemble de cartes prêtes, qui alimentent la colonne « À faire (sprint) ».

5.3 Revue de code et Review de sprint

La colonne « À review » matérialise la revue par les pairs.

5.4 Rétrospective

En fin de sprint, l'équipe identifie ce qui a fonctionné, ce qui peut être amélioré et définit des actions concrètes

5.5 Pratiques d'ingénierie

- Versionning : commits fréquents et messages en français ou anglais courts.
- Revue de code : colonnne « À review » Trello + pull requests GitHub.
- Definition of Done : une carte est « Terminé » quand elle est codée, testée, intégrée et documentée.

6. Conclusion

Le projet TP_Agile démarre sur des bases méthodologiques solides : un README détaillé, une architecture multi-agents explicite, un board Trello structuré et un dépôt GitHub versionné. La méthode Scrumban retenue est pertinente pour ce type de projet étudiant combinant cadrage agile et briques techniques incertaines.